

Cours Python

Formation Youtube



Contenu

Récapitulatif de la partie 1 : introduction	4
Principales définitions	4
Les principaux usages du Python	4
Fonctionnement des cellules dans Jupyter Notebook	5
Barre d'accès rapide Jupyter	5
Récapitulatif de la partie 2 : Les bases de Python	6
Déclaration de Variable	6
Règles de Nommage des Variables	6
Commentaires	6
Types de Variables de base	6
Opérateurs Mathématiques Basiques	6
Concaténation de Strings	6
Échapper un Caractère	7
Méthodes Courantes pour les Strings	7
Récapitulatif de la partie 3 : Les conditions	8
Variables de Type Booléen	8
Opérateurs de Comparaison	8
Conditions if, elif, else	8
Short Hand if ... else (Opérateurs ternaires)	9
If Imbriqués (Nested if)	9
Récapitulatif de la partie 4.1 : Les listes	10
Déclaration de Liste	10
Accès et Modification des Éléments d'une Liste	10
Slicing (Découpage) de Listes	10
Listes Multidimensionnelles	10
Méthodes des Listes	10
Quelques exemples pratiques (non exhaustifs)	11
Récapitulatif de la partie 5 : Les dictionnaires	12
Définition et Particularité des Dictionnaires	12
Déclaration d'un Dictionnaire	12
Accès aux Données d'un Dictionnaire	12
Modification d'un Dictionnaire	12
Suppression des Valeurs d'un Dictionnaire	12
Les Dictionnaires Imbriqués	12
Autres Méthodes Liées aux Dictionnaires	12
Récapitulatif de la partie 6 : Les boucles	13
Boucle while	13
break, continue, else	13
Opérateurs de Réaffectation	13
Boucle for	13
Boucle for sur Liste, Tuple, Dictionnaire	13
Fonction range()	13
Méthodes enumerate et zip	14
Compréhensions de Listes	14

Récapitulatif de la partie 7 : Les fonctions.....	15
Définition et Déclaration d'une Fonction	15
Paramètres de Fonctions	15
Return	15
Portées Globales et Locales des Variables	15
Récapitulatif de la partie 8 : Les packages	16
Introduction à la Notion de Package	16
Importer un Module Spécifique d'un Package	16
Utiliser des Alias	16
PIP et ses Fonctionnalités	16
Conseils d'utilisation	16
Récapitulatif de la partie 9 – Ouverture de documents	17
Ouverture et Manipulation de Document	17
Manipulation JSON	17
Requests	17
CSV	18
Récapitulatif de la partie 10 – Gestion des erreurs.....	19
Différence entre Erreurs et Exceptions	19
Try et Except	19
Les Différents Except Possibles	19

Principales définitions

- Python : Langage de programmation interprété et de haut niveau, Python est apprécié pour sa syntaxe simple et sa grande lisibilité, et il est utilisé dans divers domaines comme la science des données, le développement web, et l'automatisation.
- Kernel : Un kernel est un programme qui exécute le code contenu dans un notebook Jupyter, interprétant le code écrit dans celui-ci.
- Paquets : Les paquets sont des ensembles de modules Python qui offrent des fonctionnalités supplémentaires, installables et gérables via des gestionnaires de paquets comme pip.
- Environnements : Un environnement est un espace isolé possédant sa propre installation de Python et de paquets, permettant une gestion indépendante des configurations de projet.
- Jupyter Notebook : Jupyter Notebook est une application web permettant de créer et de partager des documents contenant du code interactif, des visualisations, et du texte narratif.
- Anaconda : Anaconda est une distribution open source des langages Python et R, principalement utilisée pour la science des données et le machine learning, facilitant la gestion des paquets et des environnements.

Les principaux usages du Python

- Développement Web : Python permet de créer des sites web grâce à des frameworks comme Django et Flask, en offrant une syntaxe claire, une sécurité renforcée et une grande adaptabilité.
- Analyse de données et science des données : Python est privilégié pour l'analyse et la visualisation de données avec des bibliothèques telles que Pandas, NumPy, et Matplotlib, offrant un écosystème adapté aux grands ensembles de données et une intégration facile avec les bases de données.
- Apprentissage automatique et intelligence artificielle : Avec des outils comme TensorFlow, PyTorch, et Scikit-learn, Python est utilisé pour construire des modèles d'IA et de machine learning grâce à sa flexibilité, ses performances, et sa grande communauté de développeurs.
- Automatisation et scripting : Python est fréquemment employé pour écrire des scripts automatisant des tâches répétitives, bénéficiant d'une vaste bibliothèque standard, et d'une portabilité entre différentes plateformes.

Fonctionnement des cellules dans Jupyter Notebook

- Cellules de code : C'est ici que vous écrivez et exécutez votre code. Les cellules de code ont une barre à gauche indiquant si elles ont été exécutées et, si oui, leur ordre d'exécution dans le notebook.
- Cellules markdown : Utilisées pour ajouter du texte formaté, des images, des liens et des listes. Le markdown est un langage de balisage léger qui vous permet de styliser votre texte facilement.

Barre d'accès rapide Jupyter

- Save (Sauvegarder) : Sauvegarde rapidement le notebook.
- + (Ajouter une cellule) : Ajoute une nouvelle cellule sous la sélection.
- Cut (Couper) : Coupe la cellule sélectionnée.
- Copy (Copier) : Copie la cellule sélectionnée.
- Paste (Coller) : Colle la cellule coupée ou copiée.
- Up Arrow (Flèche vers le haut) : Déplace la cellule sélectionnée vers le haut.
- Down Arrow (Flèche vers le bas) : Déplace la cellule sélectionnée vers le bas.
- Run (Exécuter) : Exécute la cellule et passe à la suivante.
- Stop (Arrêter) : Arrête l'exécution de la cellule.
- Restart the Kernel and Clear All Outputs : Redémarre le kernel et efface les résultats.
- Cell Type Dropdown (Type de cellule) : Change le type de cellule (code, markdown, raw).

Déclaration de Variable

- Syntaxe: `nom_variable = valeur`
- Exemple: `age = 21`

Règles de Nommage des Variables

- Doit commencer par une lettre (a-z, A-Z) ou un underscore (`_`)
- Peut contenir des lettres, des chiffres (0-9), ou des underscores
- Ne peut pas commencer par un chiffre
- Ne doit pas contenir de caractères spéciaux (sauf `'_'`)
- Ne doit pas être un mot réservé en Python (comme `if`, `else`, `for`, etc.)

Commentaires

Utilisés pour expliquer le code, rendre le code plus lisible et empêcher l'exécution de certaines lignes de code.

- Commentaire sur une ligne : Utilisez `#` au début de la ligne.
- Commentaires multilignes : Utilisez `"""` ou `'''` au début et à la fin du commentaire.

Types de Variables de base

- `int`: Nombres entiers (ex: `5`, `-3`)
- `float`: Nombres à virgule (ex: `3.14`, `-0.001`)
- `str`: Chaînes de caractères (ex: `"Bonjour"`, `'Python'`)
- `bool`: booléen (ex: `True`, `False`)

Pour modifier le type d'une variable, vous pouvez utiliser les méthodes `int()`, `float()`, `str()`

Opérateurs Mathématiques Basiques

- Addition: `+` (ex: `3 + 4` donne `7`)
- Soustraction: `-` (ex: `5 - 2` donne `3`)
- Multiplication: `*` (ex: `3 * 4` donne `12`)
- Division: `/` (ex: `8 / 4` donne `2.0`)
- Division Entière: `//` (ex: `8 // 3` donne `2`)
- Modulo (Reste de la Division): `%` (ex: `8 % 3` donne `2`)
- Exponentiation: `**` (ex: `2 ** 3` donne `8`)

Concaténation de Strings

- Avec `+`: `"Hello" + " " + "World"` donne `"Hello World"`
- Avec Literals: `f'{prenom} {nom}'` (ex: `prenom = "John"`; `nom = "Doe"` donne `"John Doe"`)

Échapper un Caractère

- Utilisez le backslash (\) pour échapper des caractères spéciaux dans une chaîne.
- Exemple: "Il dit: \"Bonjour\""

Méthodes Courantes pour les Strings

- `len()`: Retourne la longueur d'une chaîne (ex: `len("Python")` donne 6)
- `lower()`: Convertit la chaîne en minuscules (ex: `"PYTHON".lower()` donne `"python"`)
- `upper()`: Convertit la chaîne en majuscules (ex: `"python".upper()` donne `"PYTHON"`)
- `strip()`: Enlève les espaces au début et à la fin (ex: `" python ".strip()` donne `"python"`)
- `replace(old, new)`: Remplace une sous-chaîne par une autre (ex: `"python".replace("y", "i")` donne `"pithon"`)
- `split(separator)`: Divise la chaîne en liste, en utilisant le séparateur (ex: `"a,b,c".split(",")` donne `['a', 'b', 'c']`)

Variables de Type Booléen

Définition : Peuvent prendre deux valeurs : True ou False.

```
a = True
b = False
```

Opérateurs de Comparaison

- Égal à (==) : Vérifie si deux valeurs sont égales.
- Différent de (!=) : Vérifie si deux valeurs ne sont pas égales.
- Supérieur à (>) : Vérifie si la valeur de gauche est plus grande que celle de droite.
- Inférieur à (<) : Vérifie si la valeur de gauche est plus petite que celle de droite.
- Supérieur ou égal à (>=) : Vérifie si la valeur de gauche est supérieure ou égale à celle de droite.
- Inférieur ou égal à (<=) : Vérifie si la valeur de gauche est inférieure ou égale à celle de droite.
- And (and) : Vérifie si deux conditions ou plus sont toutes vraies.
- Or (or) : Vérifie si au moins une parmi plusieurs conditions est vraie.
- Not (not) : Inverse l'état de la condition (rend True si la condition est False et vice-versa).

Conditions if, elif, else

- if : Exécute le bloc de code si la condition est vraie.

```
if condition:
    # bloc de code
```

- elif : Exécute un bloc de code si la condition précédente n'est pas vraie et que cette condition est vraie.

```
if condition1:
    # bloc de code
elif condition2:
    # bloc de code
```

- else : Exécute un bloc de code si aucune des conditions précédentes n'est vraie.

```
if condition:
    # bloc de code
else:
    # bloc de code
```

Short Hand if ... else (Opérateurs ternaires)

Permet d'écrire des conditions en une seule ligne.

Syntaxe :

```
variable = valeur1 if condition else valeur2
```

If Imbriqués (Nested if)

Utilisez un if à l'intérieur d'un autre if pour tester des conditions supplémentaires.

Exemple :

```
if condition1:  
    if condition2:  
        # bloc de code
```

Déclaration de Liste

- Créer une liste : `ma_liste = [1, 2, 3]`
- Liste vide : `liste_vide = []`

Accès et Modification des Éléments d'une Liste

- Accéder à un élément : `element = ma_liste[index]`
- Modifier un élément : `ma_liste[index] = nouvelle_valeur`

Slicing (Découpage) de Listes

- Accéder à une sous-liste : `sous_liste = ma_liste[debut:fin]`

Listes Multidimensionnelles

- Liste de listes : `liste_multi = [[1, 2], [3, 4]]`
- Accéder à un élément : `element = liste_multi[i][j]`

Méthodes des Listes

- `append(element)` : Ajoute un élément à la fin.
- `insert(index, element)` : Insère un élément à la position spécifiée.
- `remove(element)` : Supprime la première occurrence de l'élément.
- `pop([index])` : Supprime et retourne l'élément à l'index (dernier par défaut).
- `del ma_liste[index]` : Supprime l'élément à l'index spécifié.
- `clear()` : Vide la liste.
- `len(ma_liste)` : Retourne le nombre d'éléments.
- `count(element)` : Compte les occurrences de l'élément.
- `sum(ma_liste)` : Retourne la somme des éléments.
- `index(element)` : Trouve l'index de la première occurrence de l'élément.
- `element in ma_liste` : Vérifie si l'élément est dans la liste.
- `sort()` : Trie la liste en place.
- `sorted(ma_liste)` : Retourne une nouvelle liste triée.
- `reverse()` : Inverse la liste en place.
- `copy()` : Crée une copie superficielle de la liste.

Quelques exemples pratiques (non exhaustifs)

```
# Ajout et suppression
ma_liste.append(4)
ma_liste.insert(2, 5)
ma_liste.remove(5)
dernier = ma_liste.pop()

# Tri et inversion
ma_liste.sort()
ma_liste_inverse = sorted(ma_liste, reverse=True)
ma_liste.reverse()

# Copie
copie_liste = ma_liste.copy()
```

Définition et Particularité des Dictionnaires

- Définition : Un dictionnaire est une collection non ordonnée, modifiable et indexée de paires clé-valeur.
- Particularité : Les clés sont uniques dans un dictionnaire et ne peuvent pas être dupliquées.

Déclaration d'un Dictionnaire

- Dictionnaire Vide : `mon_dict = {}` ou `mon_dict = dict()`
- Dictionnaire avec Paires Clé-Valeur : `mon_dict = {"clé1": "valeur1", "clé2": "valeur2"}`

Accès aux Données d'un Dictionnaire

- Accès à une Valeur : `valeur = mon_dict["clé"]`
- `keys()` : Retourne toutes les clés : `cles = mon_dict.keys()`
- `values()` : Retourne toutes les valeurs : `valeurs = mon_dict.values()`
- `items()` : Retourne toutes les paires clé-valeur : `paires = mon_dict.items()`

Modification d'un Dictionnaire

- Ajout/Modification : `mon_dict["nouvelle_clé"] = "nouvelle_valeur"`
- Mise à Jour avec un Autre Dictionnaire : `mon_dict.update(autre_dict)`

Suppression des Valeurs d'un Dictionnaire

- Supprimer une Clé : `del mon_dict["clé"]`
- Supprimer et Retourner une Clé : `valeur = mon_dict.pop("clé")`
- Vider le Dictionnaire : `mon_dict.clear()`

Les Dictionnaires Imbriqués

- Accès et Manipulation : `valeur_interne = mon_dict["clé_ext"]["clé_int"]`

Autres Méthodes Liées aux Dictionnaires

- `copy()` : Crée une copie superficielle : `copie = mon_dict.copy()`
- `in` : Vérifie l'existence d'une clé : `"clé" in mon_dict`
- `len()` : Nombre de paires clé-valeur : `taille = len(mon_dict)`
- `fromkeys()` : Crée un dictionnaire à partir d'un ensemble de clés avec une valeur par défaut : `dict.fromkeys(["clé1", "clé2"], "valeur_default")`

Boucle while

- Définition : Répète un bloc de code tant qu'une condition est vraie.
- Déclaration :

```
while condition:  
    # instructions
```

break, continue, else

- break : Sort de la boucle immédiatement.
- continue : Passe à l'itération suivante de la boucle.
- else : Bloc exécuté après la fin de la boucle, sauf si sorti par break.

Opérateurs de Réaffectation

- += : Ajoute et réaffecte (ex. x += 1).
- -= : Soustrait et réaffecte.
- *= : Multiplie et réaffecte.
- /= : Divise et réaffecte.
- %= : Modulo et réaffecte.
- //= : Division entière et réaffecte.
- **= : Élève à la puissance et réaffecte.

Boucle for

- Définition : Itère sur les éléments d'une séquence.
- Déclaration :

```
for variable in sequence:  
    # instructions
```

Boucle for sur Liste, Tuple, Dictionnaire

- Liste : for item in [1, 2, 3]:
- Tuple : for item in (1, 2, 3):
- Dictionnaire :
 - Clés : for key in dict:
 - Valeurs : for value in dict.values():
 - Clés et valeurs : for key, value in dict.items():

Fonction range()

- Utilisation : Génère une séquence de nombres.
- Exemples :
 - range(fin) : De 0 à fin - 1.

- `range(début, fin)` : De début à fin - 1.
- `range(début, fin, pas)` : De début à fin - 1, avec un intervalle de pas.

Méthodes enumerate et zip

- `enumerate` : Ajoute un compteur aux itérations.
 - Exemple : `for index, value in enumerate(sequence):`
- `zip` : Itère simultanément sur plusieurs séquences.
 - Exemple : `for item1, item2 in zip(sequence1, sequence2):`

Compréhensions de Listes

- Définition : Syntaxe concise pour créer des listes.
- Syntaxe : `[expression for item in iterable if condition]`.
- Exemple : `[x*2 for x in range(5)]` -> `[0, 2, 4, 6, 8]`

Définition et Déclaration d'une Fonction

- Définition : Un bloc de code réutilisable conçu pour effectuer une tâche spécifique.
- Syntaxe :

```
def nom_fonction(param1, param2, ...):  
    # Bloc de code  
    return resultat # Optionnel
```

Paramètres de Fonctions

- Obligatoires: Doivent être fournis lors de l'appel de la fonction.

```
def fonction(param_obligatoire):  
    # Bloc de code
```

- Optionnels: Ont des valeurs par défaut et ne sont pas obligatoires.

```
def fonction(param_optionnel=42):  
    # Bloc de code
```

Return

- Utilisation: Pour renvoyer une valeur de la fonction à l'appelant.
- Syntaxe:

```
return valeur
```

Portées Globales et Locales des Variables

- Globale: Définie en dehors de toute fonction, accessible partout dans le script.
- Locale: Définie à l'intérieur d'une fonction, accessible uniquement dans cette fonction.
- Accès Global dans une fonction:

```
global variable_globale # Référence une variable globale
```

Introduction à la Notion de Package

Un package en Python est un dossier qui contient des modules Python. Ces modules peuvent être des fichiers .py contenant des définitions de fonctions, de classes et de variables, ainsi que des instructions exécutables. Importer un package signifie rendre ces modules ou fonctions disponibles dans votre espace de travail actuel.

```
import math
```

Importer un Module Spécifique d'un Package

```
from math import sqrt, cos, sin
```

Utiliser des Alias

```
import numpy as np  
from math import sqrt as racine
```

PIP et ses Fonctionnalités

- Lister les packages installés : `PIP list`
- Afficher les informations d'un package : `pip show nom_du_package`
- Installer un package : `pip install nom_du_package`
- Mettre à jour un package : `pip install --upgrade nom_du_package`
- Désinstaller un package : `pip uninstall nom_du_package`

Conseils d'utilisation

- Environnements Virtuels : utiliser des environnements virtuels pour isoler les dépendances des projets.
- Gestion des Versions : Soyez attentifs aux versions des packages lors de l'installation pour éviter les incompatibilités.

Ouverture et Manipulation de Document

- Ouverture de Fichier
 - Lecture: with open('fichier.txt', 'r') as f:
 - Écriture: with open('fichier.txt', 'w') as f:
 - Ajout: with open('fichier.txt', 'a') as f:
- Lecture de Contenu
 - Tout lire: contenu = f.read()
 - Lire une ligne: ligne = f.readline()
 - Lire toutes les lignes: lignes = f.readlines()
 - Boucler sur chaque ligne: for ligne in f:
- Suppression de Document
 - `os.remove('fichier.txt')`

Manipulation JSON

- Équivalences JSON/Python
 - Objet JSON → Dictionnaire Python
 - Tableau JSON → Liste Python
 - String JSON → Chaîne de caractères Python
 - Nombre JSON → Int/Float Python
 - `true/false` JSON → `True/False` Python
 - `null` JSON → `None` Python
- Lecture et Écriture JSON
 - Lire JSON: `data = json.load(f)`
 - Écrire JSON (objet → fichier): `json.dump(data, f, indent=4)`
 - Convertir objet Python en chaîne JSON: `chaine = json.dumps(data, indent=4)`

Requests

- Envoyer une Requête GET
 - `reponse = requests.get('https://exemple.com')`
 - Accéder au contenu: `contenu = reponse.text` ou `response.content`

CSV

- Lire un Fichier CSV

```
with open('fichier.csv', 'r') as f:
    lecteur = csv.reader(f)
    en_tete = next(lecteur)
    for ligne in lecteur:
        print(ligne)
```

- Écrire dans un Fichier CSV

- Écrire une ligne:

```
with open('fichier.csv', 'w', newline='') as f:
    ecrivain = csv.writer(f)
    ecrivain.writerow(['Col1', 'Col2', 'Col3'])
```

- Écrire plusieurs lignes:

```
with open('fichier.csv', 'w', newline='') as f:
    ecrivain = csv.writer(f)
    ecrivain.writerows([[ 'Val1', 'Val2', 'Val3'], [ 'Val4', 'Val5',
'Val6']])
```

- Isoler l'Entête des Colonnes

- Utiliser `next(lecteur)` pour passer la première ligne d'en-tête lors de la lecture.

Différence entre Erreurs et Exceptions

- Erreurs de Syntaxe : Problèmes dans le code qui empêchent Python de l'interpréter correctement. Exemples : oublis de deux-points, indentation incorrecte.
- Exceptions : Erreurs détectées pendant l'exécution du programme, même si la syntaxe est correcte. Peuvent être gérées et récupérées.

Try et Except

- try : Bloc de code à surveiller pour les exceptions. Si une exception survient, le flux est transféré au bloc except.

```
try:  
    # Code qui pourrait lever une exception
```

- except : Spécifie un type d'exception à capturer et à gérer.

```
except TypeError:  
    # Traitement spécifique pour TypeError  
except Exception as e:  
    # Traitement pour toutes les autres exceptions capturées
```

Les Différents Except Possibles

- Spécifiques : Capturer des types d'exceptions précis (e.g., ValueError, ZeroDivisionError).
- Générique : Utiliser except Exception as e pour capturer toute exception non capturée par les blocs except précédents.
- Tout Capturer : except: sans spécifier un type capture toutes les exceptions, y compris les exceptions système. À utiliser avec prudence.